

Auditoria de Seguranca

Alivia Finance — PWA de finanzas pessoais (soualivia.com.br)
Data: 08/06/2026 | Escopo: revisao somente leitura (nenhuma alteracao aplicada)
Referencias: OWASP Top 10, OWASP ASVS, OWASP API Security Top 10, CWE Top 25, NIST SSDF, Firebase / Stripe / Google OAuth / Vercel Security Best Practices.

Resumo executivo

Revisao de firestore.rules, AuthContext.jsx, api/*, vercel.json, vite.config, service worker, .env e stripe.js, mais npm audit.

Severidade	Qtde	Itens
Critica	1	F-01
Alta	4	F-02, F-03, F-04, F-05
Media	4	F-06, F-07, F-08, F-09
Baixa / Info	5	F-10 a F-14

Resposta a pergunta-chave do checkout (e possivel alterar valores/planos/permissoes durante o pagamento?): o endpoint de upgrade (/api/upgrade-subscription) esta **bem protegido** (valida ID token + allowlist de preco + so altera a propria assinatura). **Porem SIM, ha dois caminhos para obter plano pago sem pagar:** (a) escalonamento direto no Firestore (F-01) e (b) preco desconhecido tratado como premium + checkout_sessions controlado pelo cliente (F-03).

Achados detalhados

CRITICA F-01 — Escalada de privilegio para plano lifetime (e admin no cliente) via regra generica do Firestore

Categoria: OWASP A01 Broken Access Control; API1/API5; CWE-639 / CWE-284
Complexidade da correcao: Media | Risco de regressao: Medio

Evidencia

Regra catch-all de colecao convive (OR logico) com a regra protegida de users:

```
match /{collectionName}/{docId} { // casa /users/{uid} tambem
  allow update: if ownsExisting() && ownsIncoming() && ownerStable();
}
match /users/{userId} { // guarda anti-promocao
  allow write: if ... && subStatusSafe() && adminFlagSafe();
}
```

O app deriva plano/admin de campos do proprio doc: AuthContext.jsx:213-214 (subscription.status == 'lifetime' => planLevel='lifetime') e :142 (userData.isAdmin == true => isAdmin). O perfil e gravado com campo 'uid' (nao 'userId'), entao um ataque em dois passos no console funciona:

```
1) updateDoc(users/<meuUid>, { userId:<meuUid> }) // passa na regra de users
2) updateDoc(users/<meuUid>, { userId:<meuUid>,
  isAdmin:true, subscription:{status:'lifetime'} }) // regra generica autoriza e
// ignora adminFlagSafe/subStatusSafe
```

Impacto: Qualquer usuario gratuito libera premium/vitalicio sem pagar. Ativa a UI Admin no cliente. Atenuante: leitura de dados de terceiros segue bloqueada (a regra isAdmin do Firestore valida e-mail do token), logo NAO ha vazamento de dados de outros usuarios nem IDOR horizontal.

Correcao recomendada: Remover o catch-all match /{collectionName}/{docId} e declarar explicitamente as 11 colecoes financeiras reais (transactions, goals, cards, subscriptions, fixed_expenses, savings_jars, investments, tangible_assets, debts, insurances, watchlist), sem sobrepor

users/customers/settings/admin_configs.

ALTA F-02 — /api/send-push sem autenticacao (broadcast para toda a base)

Categoria: OWASP API2 Broken Authentication; CWE-306

Complexidade da correcao: Baixa | **Risco de regressao:** Baixo

Evidencia

api/send-push.js nao valida token/identidade. Faz db.collection('users').get() e envia para todas as pushSubscriptions, com title/body e a URL controlados pelo corpo (linhas 39-69). O service worker abre essa URL no clique (sw.js:60-64).

Impacto: Qualquer pessoa pode disparar notificacao para todos os usuarios com texto e link arbitrarios => phishing em massa, spam e dano de reputacao.

Correcao recomendada: Exigir Authorization Bearer idToken, verifyIdToken e checar isAdminEmail(decoded.email) (mesmo padrao do upgrade-subscription.js). Opcional: allowlist do dominio da URL.

ALTA F-03 — Plano derivado de 'preco desconhecido => premium' + checkout_sessions controlado pelo cliente

Categoria: OWASP A01 / API6; CWE-840 (Business Logic)

Complexidade da correcao: Baixa (item 1) | **Risco de regressao:** Baixo

Evidencia

AuthContext.jsx:247-249: assinatura ativa com preco fora das allowlists => currentPlanLevel='premium'. stripe.js grava customers/{uid}/checkout_sessions no cliente e firestore.rules:127-128 permite escrita livre desse doc; a extensao Stripe repassa campos (incl. line_items/price_data) para a sessao.

Impacto: Usuario pode criar checkout com preco proprio (line_items/price_data com valor baixo recorrente). Ao concluir, a assinatura fica ativa com preco nao reconhecido e o app concede premium => premium por valor irrisorio (condicional a config da extensao).

Correcao recomendada: Defesa em profundidade: (1) no app, preco ativo desconhecido NAO deve conceder premium (free/standard); (2) validar no servidor que o price pertence a allowlist antes de conceder plano.

ALTA F-04 — Dependencias com CVEs conhecidas (incl. jsPDF critico em runtime)

Categoria: OWASP A06 Vulnerable & Outdated Components; CWE-1104

Complexidade da correcao: Media | **Risco de regressao:** Medio

Evidencia

npm audit: 32 vulnerabilidades (3 criticas, 10 altas). Em runtime/cliente: jspdf/jspdf-autotable (critica - LFI/Path Traversal + injecao, usada em utils/generatePDF.js) e lodash (alta - code injection via _.template). Transitiva do firebase-admin: protobufjs (critica). Demais (vite/rollup/babel/serialize-javascript/minimatch/picomatch/flatted/fast-uri) sao build-time (risco real menor).

Impacto: jsPDF processa dados financeiros no navegador; CVEs de injecao/LFI podem executar JS no contexto da geracao do PDF. As de build-time afetam a cadeia de build, nao o usuario final.

Correcao recomendada: npm audit fix; atualizar jspdf/jspdf-autotable, firebase/firebase-admin e vite. Build + smoke test de PDF e login.

ALTA F-05 — Bypass de autenticacao de DEV dependente so de variavel de ambiente

Categoria: OWASP A05 Security Misconfiguration; CWE-489 (Active Debug Code)

Complexidade da correcao: Baixa | **Risco de regressao:** Nenhum/Baixo

Evidencia

AuthContext.jsx:52,75-88: IS_DEV = VITE_USE MOCK_AUTH === 'true' cria usuario admin mockado, premium, sem login. .env.local tem VITE_USE MOCK_AUTH='true'.

Impacto: Se VITE_USE MOCK_AUTH=true existir no build de producao, todo o app vira admin sem autenticacao. Depende so de uma env var booleana.

Correcao recomendada: Reforcar o gate: exigir tambem `import.meta.env.DEV === true` e/ou `hostname localhost`; garantir que a `env` nao existe no projeto de producao da Vercel.

MEDIA F-06 — CSP com unsafe-inline e unsafe-eval em script-src

Categoria: OWASP A05; CWE-1021 / CWE-79

Complexidade da correcao: Media | **Risco de regressao:** Medio

Evidencia

`vercel.json`: `script-src 'self' 'unsafe-inline' 'unsafe-eval' ...`. Tokens do Firebase ficam em IndexedDB e a chave Gemini em `localStorage` (F-08), acessiveis a qualquer XSS.

Impacto: Qualquer injecao de HTML/JS vira XSS executavel (incl. roubo de ID token / chave Gemini). React mitiga, mas a CSP deixa de ser a 2a barreira.

Correcao recomendada: Remover `unsafe-inline/unsafe-eval` do `script-src` (usar `nonce/hash`); validar libs que exigem `eval`. Testar `Stripe.js`/`Firebase`/`PWA` apos apertar.

MEDIA F-07 — APIs de mercado publicas sem autenticacao, rate limit ou limites de entrada

Categoria: OWASP API4 Unrestricted Resource Consumption; CWE-770

Complexidade da correcao: Baixa | **Risco de regressao:** Baixo

Evidencia

`api/prices.js`, `quotes.js`, `history.js`, `fipe.js`, `tesouro.js`, `benchmarks.js`: GET publicos que fazem fan-out de fetches externos a partir de `symbols/groups` sem limite de quantidade nem `rate limiting`; usam proxies de terceiros (`corsproxy.io`, `allorigins`).

Impacto: Abuso/amplificacao: muitos simbolos por request geram muitas chamadas externas (custo/abuso de funcao e possivel `rate-limit/ban` das fontes). SSRF e baixo (`hosts` fixos, `encodeURIComponent` aplicado).

Correcao recomendada: Limitar nro de simbolos por request (ex.: 25), validar formato do ticker por regex e aplicar `rate limiting` leve por IP.

MEDIA F-08 — Chave Gemini (BYOK) armazenada em texto plano (localStorage + Firestore)

Categoria: OWASP A02 Cryptographic Failures; CWE-522 / CWE-312

Complexidade da correcao: Media/Alta | **Risco de regressao:** Medio

Evidencia

`AiChat.jsx:166`, `FinancialAdvisor.jsx:102`, `SettingsTab.jsx:78`: `localStorage.user_gemini_api_key` e `userPrefs.apiKey/manualConfig.geminiKey` em claro. E a chave do proprio usuario (sem segredo compartilhado do app exposto, o que e positivo).

Impacto: Exposicao a XSS (F-06) e a quem tiver acesso ao dispositivo. No Firestore fica restrita ao dono (regras OK), mas em claro.

Correcao recomendada: Documentar o risco; idealmente intermediar chamadas Gemini por um proxy server-side (a chave nao trafega/armazena no cliente). No minimo, nao logar a chave.

MEDIA F-09 — Regras do Firestore sem validacao de schema/campos

Categoria: OWASP API8 / A04 Insecure Design; CWE-20

Complexidade da correcao: Media | **Risco de regressao:** Medio

Evidencia

As regras das colecoes financeiras validam propriedade (`userId == uid`) mas nao tipos/limites de campos. Combinada com o `catch-all` (F-01), permite gravar campos arbitrarios e ate criar docs em colecoes inesperadas.

Impacto: Integridade/uso indevido de dados (poluicao de documentos, storage gratuito). Sem impacto de confidencialidade de terceiros.

Correcao recomendada: Apos F-01, adicionar validacoes minimas (ex.: `amount is number`, tamanho de strings) onde fizer sentido. Incremental.

F-10 a F-14 — Baixa / Informativa

ID	Item	Nota
F-10	console.log de dados financeiros/assinatura em producao (AuthContext.jsx:193-194)	CWE-532. Visível só no console do próprio usuário, mas pode cair em ferramentas de erro. Remover/mascarar.
F-11	CSRF não aplicável (positivo) - auth por Bearer token, não cookie	Manter assim. Risco real de roubo de token e via XSS (F-06).
F-12	Lista de admins hardcoded em firestore.rules e constants/admins.js	Aceitável; admin do Firestore e por e-mail do token (forte). Manter centralizado e revisar.
F-13	CSP img-src https: (qualquer host) e connect-src com corsproxy.io/allorigins	Baixo - só símbolos de mercado trafegam por proxies. Restringir img-src se possível.
F-14	LGPD: dados escopados por usuário; terms_log imutável (bom). deleteAccount mantém doc users marcado deleted com e-mail (retenção)	Documentar retenção na política; oferecer expurgo definitivo sob solicitação.

Pontos positivos confirmados

HSTS preload, X-Frame DENY, nosniff, Referrer-Policy, Permissions-Policy e CSP presentes (vercel.json); HTTPS forçado (upgrade-insecure-requests); sem source maps em produção; service worker NAO cacheia respostas de API/Firestore (sem dados financeiros offline); .env fora do git; /api/upgrade-subscription válida token + allowlist de preço + escopo do próprio usuário; coleções financeiras sem IDOR horizontal.

Plano de ação

A) Correções sem risco / baixo risco (recomendado fazer já)

- F-02 - Exigir ID token + admin em /api/send-push. (Baixa / Baixo)
- F-03(1) - AuthContext: preço ativo desconhecido NAO concede premium (free/standard). (Baixa / Baixo)
- F-05 - Gate do mock-auth também por import.meta.env.DEV/localhost + remover env de produção. (Baixa / Nenhum)
- F-07 - Limitar nro de símbolos + regex de ticker nas APIs de mercado. (Baixa / Baixo)
- F-10 - Remover/mascarar console.log de dados sensíveis. (Baixa / Nenhum)
- F-04 (parcial) - npm audit fix não-breaking + atualizar jspdf. (Baixa-Media / Baixo-Medio)

B) Correções com impacto maior / exigem teste cuidadoso (planejar)

- F-01 (Crítica) - Reescrever regras: trocar catch-all por allowlist explícita das 11 coleções, sem sobrepor users/customers. (Media / Medio - testar todas as telas).
- F-06 - Endurecer CSP (remover unsafe-inline/unsafe-eval). (Media / Medio - testar Stripe/Firebase/PWA).
- F-08 - Proxy server-side para Gemini (mudança arquitetural). (Alta / Medio).
- F-09 - Validação de schema nas regras. (Media / Medio).
- F-04 (major) - Bumps maiores (vite/firebase-admin/jspdf) com regressão de build. (Media / Medio).

Prioridades negociáveis: F-01 (premium/admin grátis via Firestore) e F-02 (push não autenticado). Nenhuma alteração de código foi aplicada nesta auditoria.